Minimax Solver & Analysis Report

Macus Wu, Owen Levine, Law Morano, Rahul Allala

1 Introduction

In order to know for certain that our Minimax solver bot generates the best choice for each possibility, we selected to recreate all four of the games: including the Halving Game, tic-tac-toe, Connect Four, and Nim.

The halving game starts with a random amount of rocks ranging from 15–50. The first player has the option to take either one of the rocks or half of the rocks. If half of the rocks are taken when the amount of rocks left is an odd number, the program will take one away from the rocks and then divide by two from the total amount of rocks. Then, the second player gets the identical choices. Both players alternate until one player takes the last rock, which makes them the winner of the game. Tic-tac-toe requires two players to take turns putting a cross or a circle inside a 3 by 3 grid. The game can end in one player winning, where they make a line with three of their own shapes either horizontally, vertically or diagonally. The game can also end in a draw, where the grid becomes full with neither player having the desired pattern on the board. Connect Four works in a similar manner compared to tic-tac-toe, where players have to create a line of four by putting either "r" or "y" chip (which stands for red or yellow) into the provided 4 by 4 grid. The lines can be horizontal, vertical or diagonal as well. However, the player can only select columns and not specific boxes, and the user's chip will be stacked on top of the highest chip in the row. If there isn't a previous chip in the selected column, the chip will be placed in the bottom of that row. Finally, the Nim game is the same as the halving game where taking the last stone wins the game. The players take turns taking from different piles of stones. There are four piles of stones: 1 stone, 3 stones, 5 stones, and 7 stones. Each player can take any amount of stones at a time, but only from one pile each time.

All four of these games are zero-sum games, which allows us to use the same minimax method for each game to find the best continuation. Zero sum games refers to where when one player gains, the other player loses. Because of this feature, we can allow the bot to figure out all the outcomes in any state of the game. This is the minimax policy our team decided to apply to our bot. By comparing our minimax bot with the other bot with a separate policy, we can analyze the performance of the minimax bot and take its effectiveness into account.

2 Strategy and Implementation

2.1 Strategy and Logic

$$V_{\text{minmax}}(s) = \begin{cases} \text{Utility}(s) & \text{if IsEnd}(s) \\ \max_{a \in \text{Actions}(s)} V_{\text{minmax}}(\text{Succ}(s,a)) & \text{if Player}(s) = +1 \\ \min_{a \in \text{Actions}(s)} V_{\text{minmax}}(\text{Succ}(s,a)) & \text{if Player}(s) = \text{-}1 \end{cases}$$

To determine the code of our function minimax, we first have to determine the separate outcomes of a zero-sum game. The logic we used as an approach here was creating minimax with a recursive

function, so that it can loop through every outcome and possibility of whatever game we provide it through the pre-existing parameters.

To start our recursive function off, we had to come up with our base case. To make the base case the most simple and ideal case, we simply said that when the game is over, we return the utility which will have the program print out the winner which will also stop the function. This base case will be satisfied once one of the players involved in the game wins.

Our second condition has one of the players wanting to maximize the number printed out in utility. In order to do that, this condition uses V_{minmax} as a recursive function to look into all the different possible outcomes by simulating each of the game's outcomes and therefore assigning a value of $+n$ or $-n$ to that equation. $+n$ would mean that this player wins the game and $-n$ would mean that this player loses the game. In this second condition, this player wants to maximize the value so that the outcome gives $+n$, which is how it chooses its conditions throughout the game.

Lastly, our third condition works similarly with our previous condition. However, instead of seeking for the maximum value onto each one of the decisions made in the game, it looks for the minimum because this player wants to minimize the game's outcome value to counteract what the previous player has done.

These conditions ensure that every move that both players play are the most optimal they can be.

2.2 Code Explanation

```
def minimax(game, state):
 if game.is_end(state):
   return (game.utility(state), None)
```

The first "if" conditional represents the base case. As described in the logic explanation, the base case detects when the game ends. It then stops the actions and returns the score of the game.

```
 results = []
 for action, next_state in game.successors(state):
   value, action_temp = minimax(game, next_state) # recurse
   results.append((value, action))
```

Our for loop iterates through every possible move you could make as `successors(state)` returns every single legal move. It sets action to a specific legal move. Following that, `next_state` is the state of the board or game after that move. We then assign a value or the result if both players play perfectly by recursively calling `mini_max` again. However, in this case, calling it on the `next_state` instead of the original state will answer the question of what the next value is. From there, we can find out if our current position is winning or losing based on the values of the successors. Our last part just adds the value we found associated with its action to a list. When the for loop finishes iterating, it will contain a list of all the possible actions at a position and the value associated with the action.

```
 if game.player(state) == 0:
   return max(results, key = get_value)
 else:
   return min(results, key = get_value)
```

Finally, the if statement here checks which player is playing. If `game.player(state) == 0`, player 0 is playing. Therefore, we use the max function and we use the "get_value" function as an enhancement to our minimax function to extract just the value from the original list of tuples so the max function can find a max value. The max expression will automatically find a move that will give the outcome value a positive value, which optimizes the chances of player 0 winning. The else statement happens

when player 1 is playing instead of playing 0. Maximum and minimums are used because the players try to counteract the actions of each other so player 0 is trying to find the maximum value while player 1 is trying to find the minimum value.

3 Simulation Design

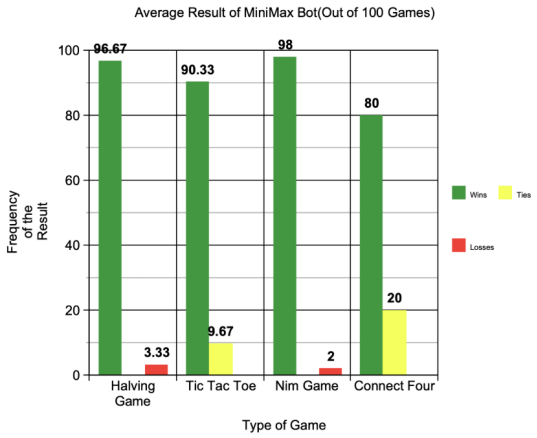
For our simulations, we used the minimax engine to create a bot that played the best moves to play against another custom bot that would play a move at random. We implemented this by finding every possible action in the position using the actions method. Then we used the random choice method to choose a random action from all the possible actions. This is the baseline for our simulations. Our code then prompts the user to enter a number of simulations. The simulation code then runs these simulations, one simulation at a time, and records the amount of times the best-move-bot wins, loses, and draws (if applicable). The bot also tracks the average game length by counting the amount of steps for each game then averaging that number. Finally, our code tracks the amount of times our minimax bot wins from a starting position versus playing second. After all simulations are completed, the results are printed in the terminal.

		MINIMAX Wins					MINIMAX Losses				
Game Name	Games Played	Trial 1	Trial 2	Trial 3	Average	STDEV	Trial 1	Trial 2	Trial 3	Average	STDEV
Halving Game	100	97	97	96	96.67	0.58	3	3	4	3.33	0.58
Tic Tac Toe	100	91	88	92	90.33	2.08	0	0	0	0.00	0.00
Nim Game	100	98	99	97	98.00	1.00	2	1	3	2.00	1.00
Connect Four	100	75	78	87	80.00	6.24	0	0	0	0.00	0.00

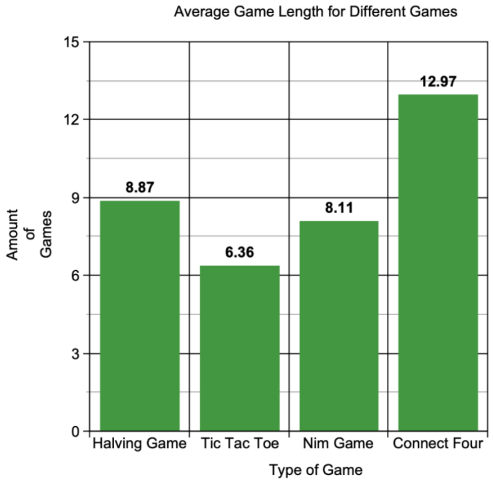
	MINIMAX Ties					Game Length				
Game Name	Trial 1	Trial 2	Trial 3	Average	STDEV	Trial 1	Trial 2	Trial 3	Average	STDEV
Halving Game	-	-	-	-	-	9.01	8.65	8.95	8.87	0.19
Tic Tac Toe	9	12	8	9.67	2.08	6.34	6.4	6.33	6.36	0.04
Nim Game	-	-	-	-	-	8.14	7.94	8.25	8.11	0.16
Connect Four	25	22	13	20.00	6.24	13.1	13.08	12.73	12.97	0.21

	MINIMAX Wins as 1st Player					MINIMAX Wins as 2nd Player				
Game Name	Trial 1	Trial 2	Trial 3	Average	STDEV	Trial 1	Trial 2	Trial 3	Average	STDEV
Halving Game	44	47	39	43.33	4.04	53	50	57	53.33	3.51
Tic Tac Toe	57	50	59	55.33	4.73	34	38	33	35.00	2.65
Nim Game	50	57	52	53.00	3.61	48	42	45	45.00	3.00
Connect Four	34	38	39	37.00	2.65	41	40	48	43.00	4.36

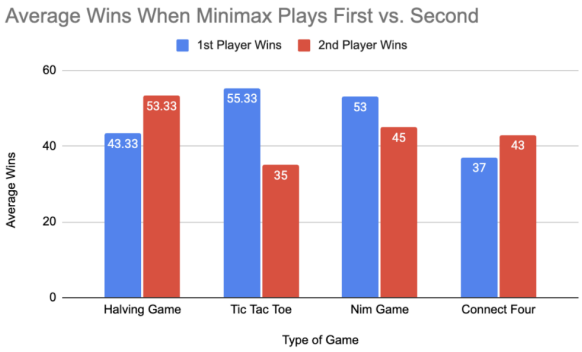
Minimax data tables: Wins, Losses, Ties, Game Length, and Wins as 1st/2nd Player.



Graph 1: Shows the difference in averages between the Wins, Losses, and Ties of the bot across the average of 100 game simulations for 4 different games.



Graph 2: Describes the average game length for all games across all 3 tournament simulations.



Graph 3: This graph compares the average wins the minimax bot has when it plays first compared to when it plays second.

4 Result Discussion

4.1 Average Wins, Losses, and Ties for the Minimax Bot

Our minimax bot was very successful picking the best move which can be seen specifically in the Halving and Nim games, where the bot only lost between 2–3% of its matches. Additionally, in tic-tac-toe and Connect Four, the bot never lost a single game, but won significantly less games, most likely because of a smaller set of move choices which granted the random move bot a greater chance at playing perfectly as well. The high win rate and zero recorded losses in tic-tac-toe and Connect Four proved that the bot accounts for opponents moves and chooses the best choice that also accounts for opponents best moves.

The losses of our minimax bot in the halving game and the Nim game can be attributed to the random bot starting with a number with a guaranteed win. In the halving game, numbers such as 17 and 29 guarantee a player's win if they play their moves optimally. According to graph 1, the staggering 96.67 average wins (out of 100) for the halving game and 98 average wins proves that our minimax

bot uses the most optimal strategy. Therefore, we can conclude that these losses are attributed to the random bot coincidentally making the correct moves when playing our minimax bot.

In Tic-tac-toe and Connect Four, the ties can be attributed to our minimax bot trying to optimize game outcome favor to themselves and choosing to tie. Tic-tac-toe specifically has a high tie rate in general because there are a limited number of choices the players can make in general. The connect four board is also usually played on a 7 by 6 board, but our version uses a 4 by 4 board, giving the random bot a larger margin of error so that it can tie with our minimax bot.

4.2 Game Length

We can see from graph 2 that Connect Four had the greatest game length, while tic-tac-toe had the smallest game length. This phenomenon can be attributed to the amount of choices the game offers to the players. Connect Four offers 16 choices for both users. tic-tac-toe, on the other hand, only offers 9 choices.

According to our second table, The game length's standard deviation ranges from 0.04–0.21. This small range of standard deviation shows that our minimax bot uses the logic explained above to make the most optimal decisions to play against the bot that does moves randomly.

4.3 Starting Position

Our results comparing the minimax bot's wins when playing first versus second show that some games favor a starting position while others do not. Randomness can affect the results if the minimax bot is selected to play first more often than second. However, we reduced this effect by running 100 simulations across three trials for each game. In the halving game, playing second was more advantageous, with an average of 53.33 wins compared to 43.33 when playing first, likely because the random bot has more opportunities to make mistakes. In tic-tac-toe, playing first provided a clear advantage (55.33 wins versus 35) by allowing the minimax bot to apply early pressure. In the Nim Game, playing first gave only a slight advantage (53 wins versus 45), suggesting the starting position has little impact. Finally, Connect Four slightly favored playing second (43 wins versus 37), possibly because moving last on a 4×4 board can provide the winning move.

4.4 Conclusion

Our minimax bot was successful in playing the best possible move in its position for each of the four games we coded. Data backs this claim up. According to table 1, our bot was able to win an average of 96.67, 90.33, 98, and 80 times across three 100 sized simulations against a random move bot, which we chose over a repeating bot because the randomness forces the minimax bot to go through a much wider variety of positions. Additionally, in strategic games like tic-tac-toe and Connect Four, our bot did not lose a single game. When given the choice, the bot will protect against losing rather than trying to win from its position which would result in a loss, which shows the bot evaluates the opposition's best move when deciding its move.

Our data also shows that the four games had different lengths; Tic-tac-toe had the shortest average game length with 6.36 moves and Connect Four had the longest average game length with 12.97 moves.

Lastly, the data collected about how the minimax prefers the starting position opposed to second position expresses that the bot prefers starting second for the Halving Game, first for tic-tac-toe, first for Nim Game, and second for Connect Four.